

capitolo 11 – Introduzione ai DB relazionali

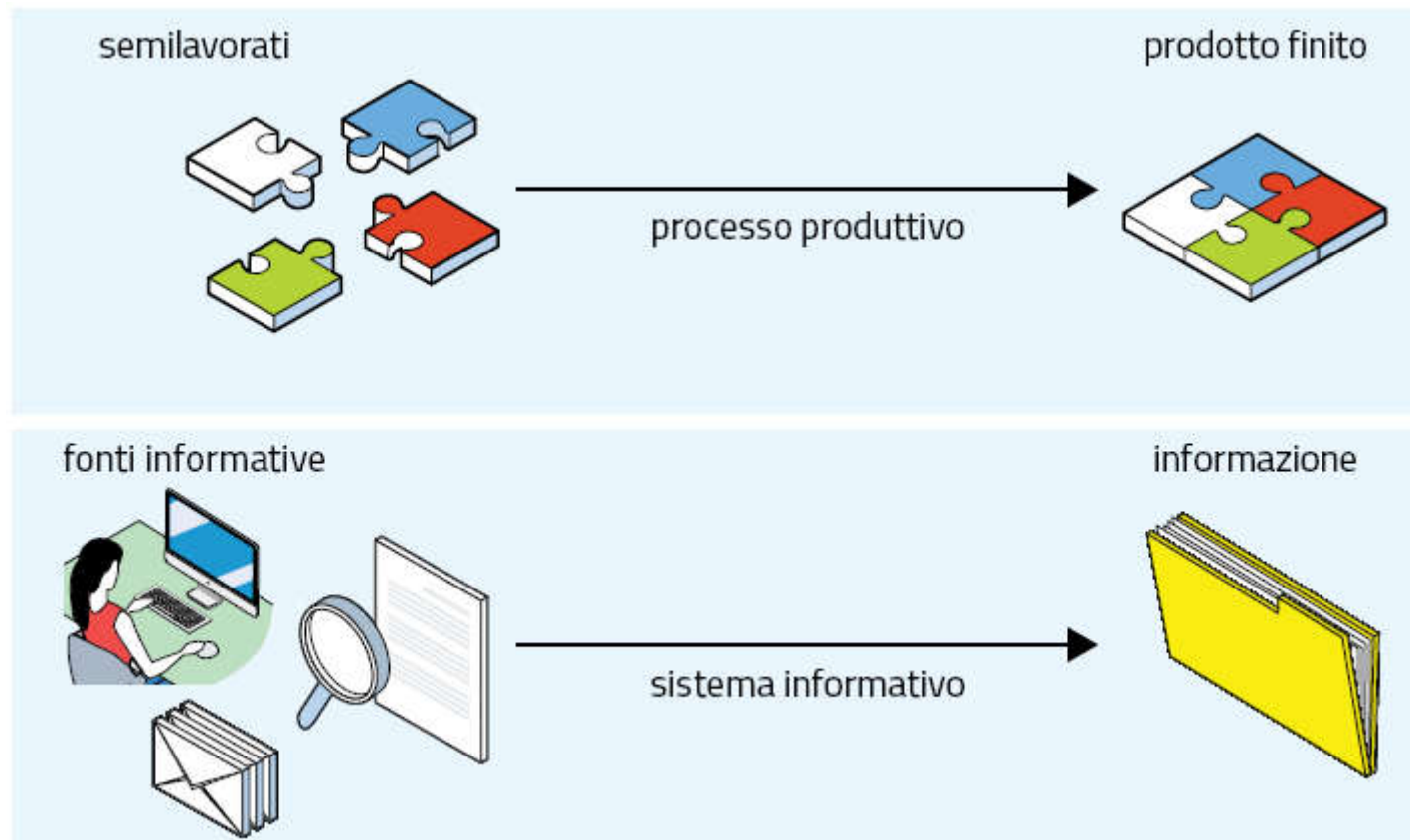
1. Sistemi informativi e sistemi informatici
2. Le basi dati e i DBMS
3. La progettazione dei database
4. Il modello logico dei dati ed il linguaggio
5. Le basi teoriche del modello relazionale
6. I valori nulli e i vincoli sui dati

11.1 Sistemi informativi e sistemi informatici

Indice

Il **sistema informativo** o **SI** ha il compito di gestire le informazioni.

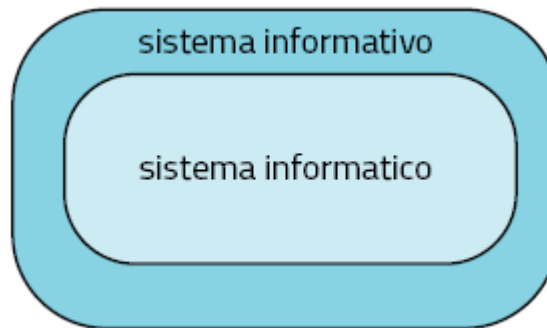
C'è analogia tra un **processo produttivo** e un **sistema informativo**:



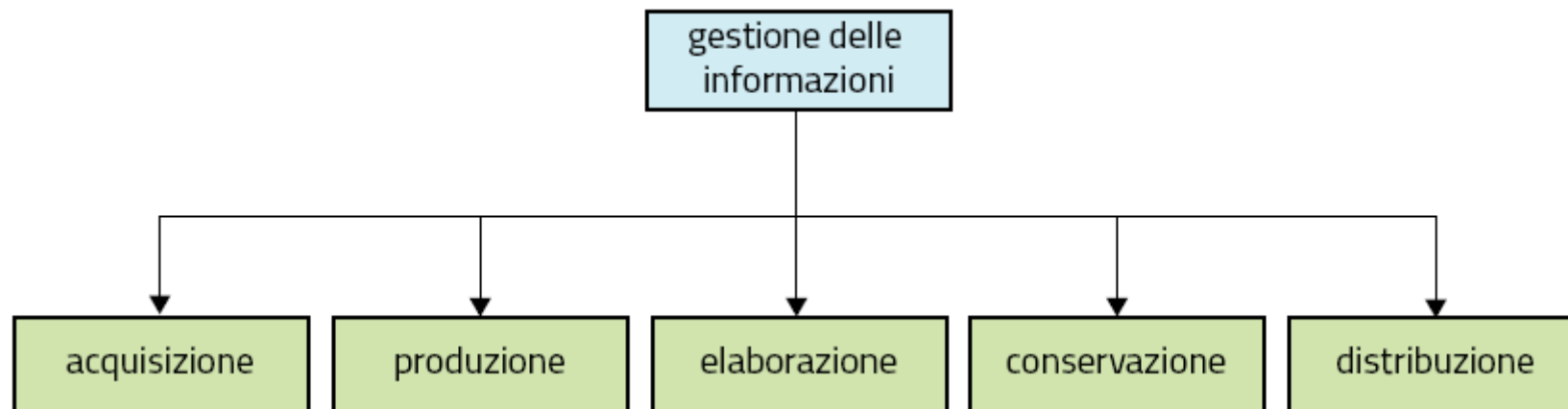
11.1 Sistemi informativi e sistemi informatici

Indice

Il **sistema informativo** o **SI** ha il compito di gestire le informazioni.
Un **sistema informatico** è la parte di un SI che è stata automatizzata.



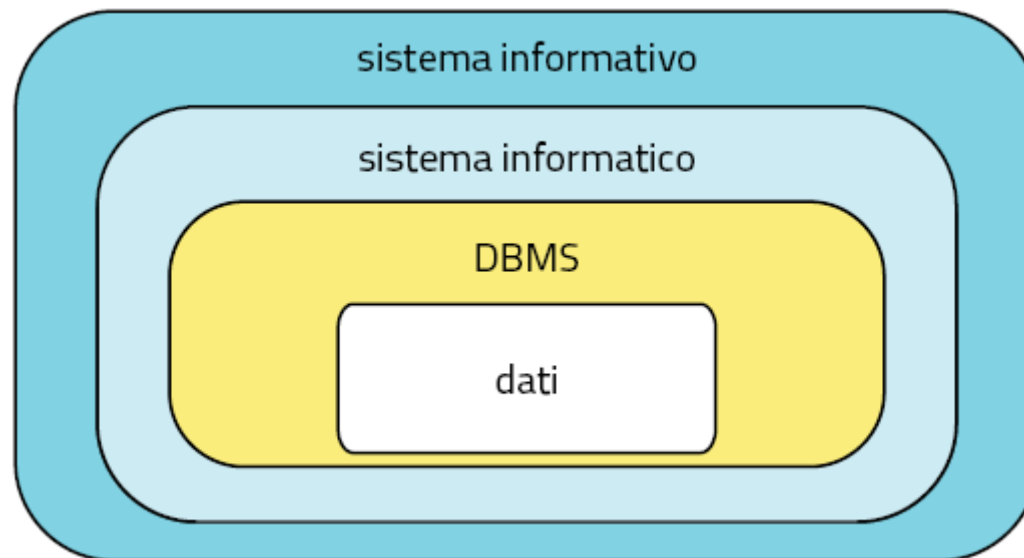
La **gestione delle informazioni** ha diverse fasi:



11.2 Le basi di dati e i DBMS

Indice

Un **DBMS** è un software che gestisce grandi quantità di dati.

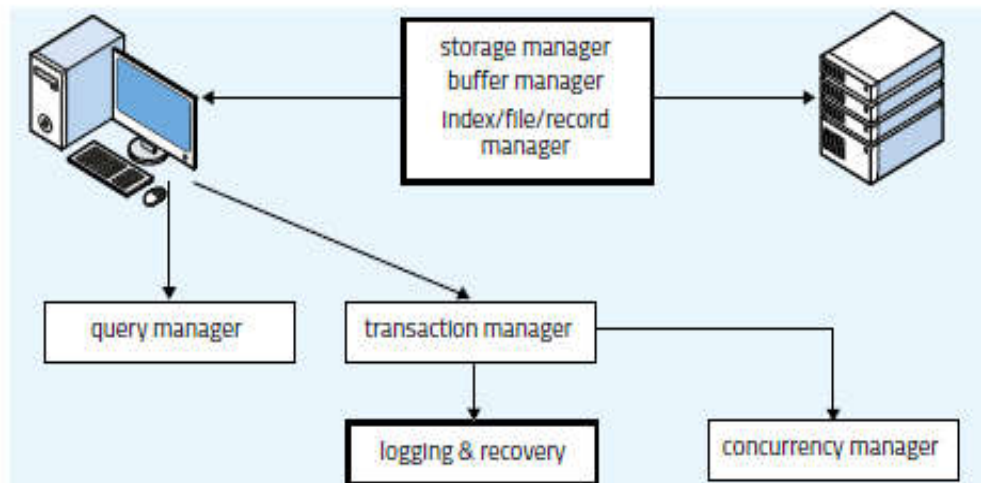
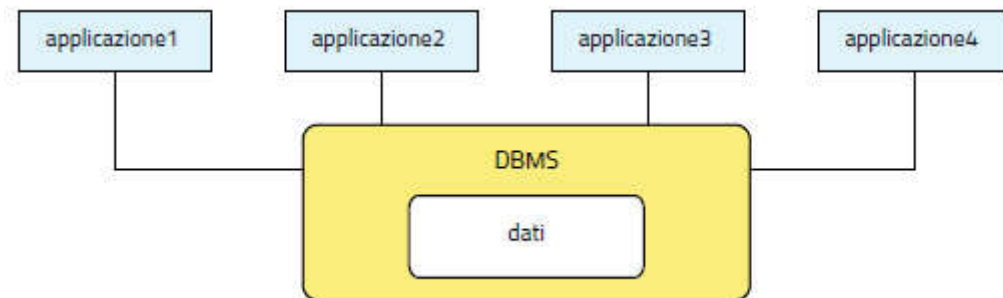


11.2 Le basi di dati e i DBMS

Indice

Un **DBMS** è un software che gestisce grandi quantità di dati.

Molte applicazioni diverse possono accedere agli stessi dati.



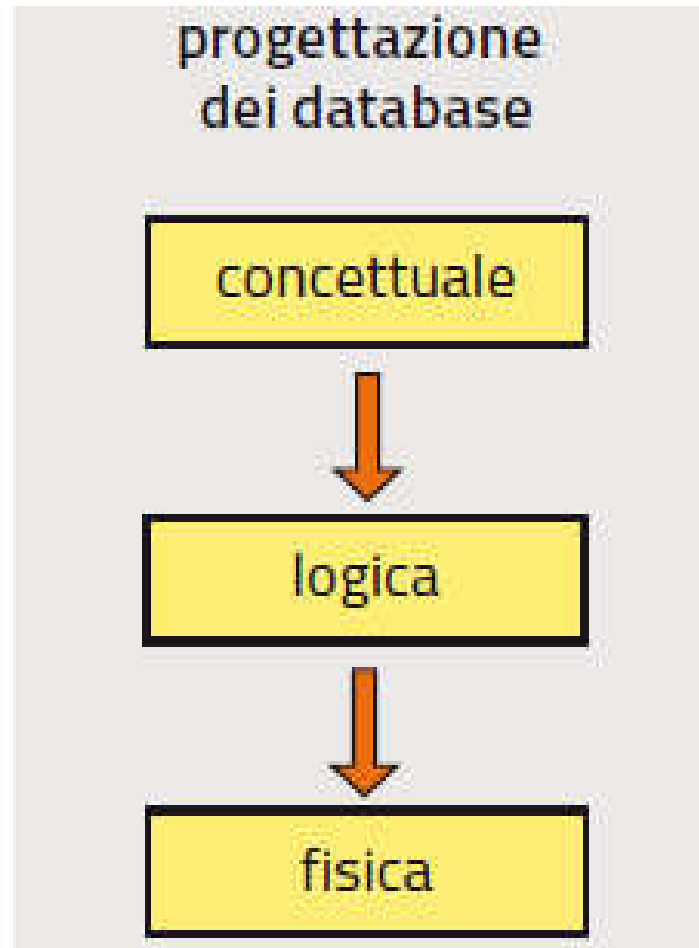
Un buon DBMS:

- assicura l'**integrità dei dati**
- evita la **ridondanza**
- controlla le **autorizzazioni**

11.3 La progettazione dei database

Indice

Il **modello E-R** è uno strumento utile nella **progettazione concettuale** dei database, che prosegue con la costruzione dei modelli logico e fisico.



11.3 La progettazione dei database

Indice

Il **modello E-R** è uno strumento utile nella **progettazione concettuale** dei database, che prosegue con la costruzione dei modelli logico e fisico.

Gli **schemi E-R** (*Entity-Relationship*) sono uno strumento per rappresentare le **entità**, cioè categorie di oggetti che hanno caratteristiche comuni, insieme alle **associazioni** che esistono tra le diverse entità.



associazione uno-a-molti

i numeretti
rappresentano
la **cardinalità**
dell'associazione

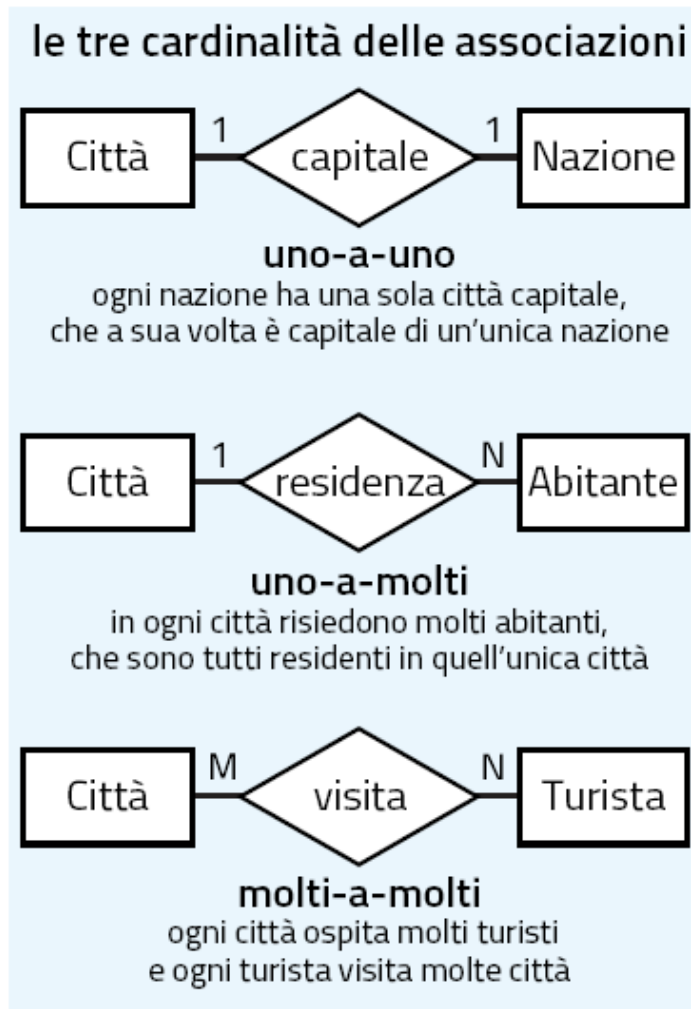


associazione multi-a-molti

11.3 La progettazione dei database

Indice

Ogni **associazione** è caratterizzata da una **cardinalità**.

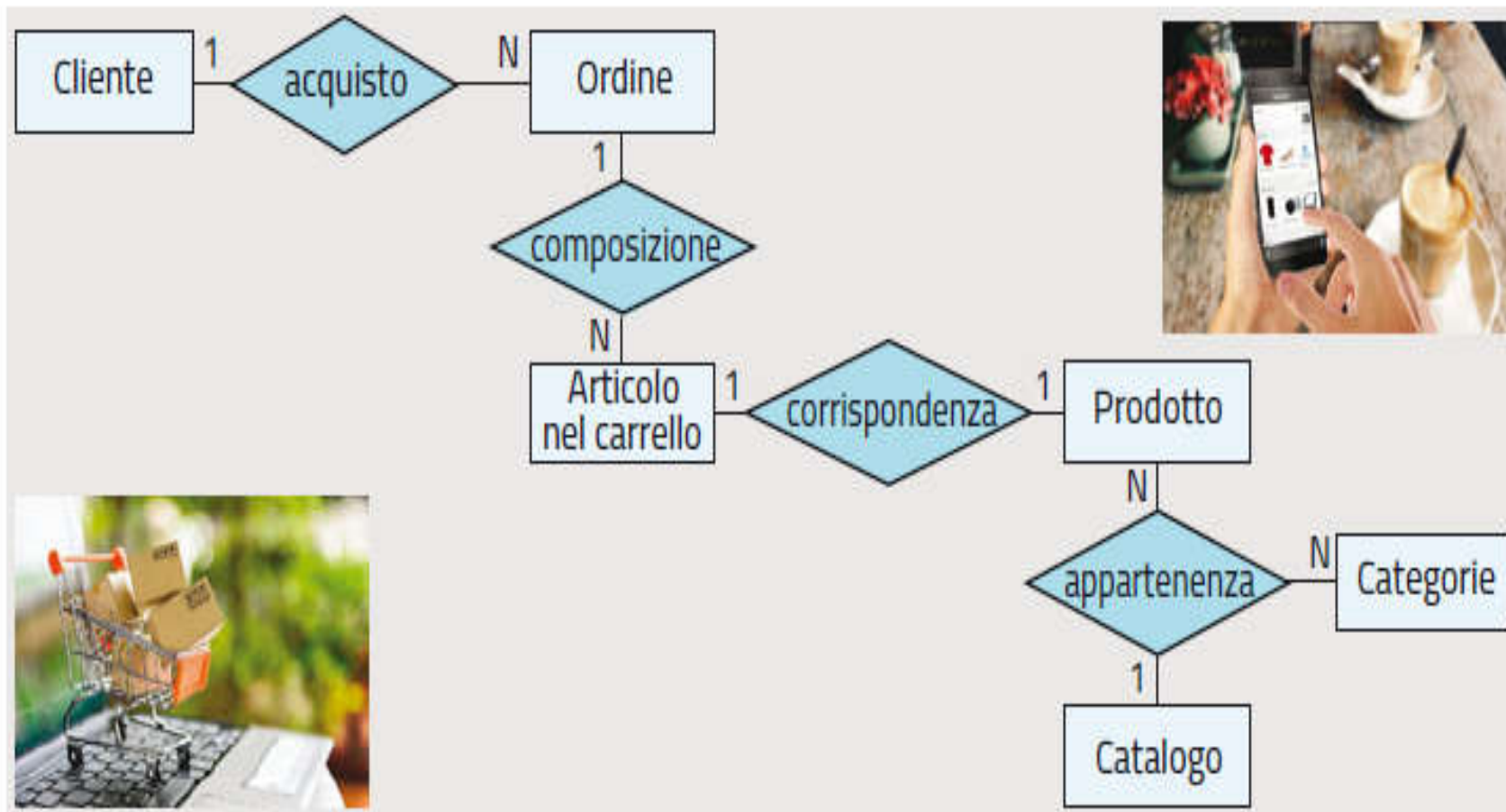


La **cardinalità** esprime il tipo di associazione, specificando quanti elementi di ciascuna entità possono essere associati all'altra.

11.3 La progettazione dei database

Indice

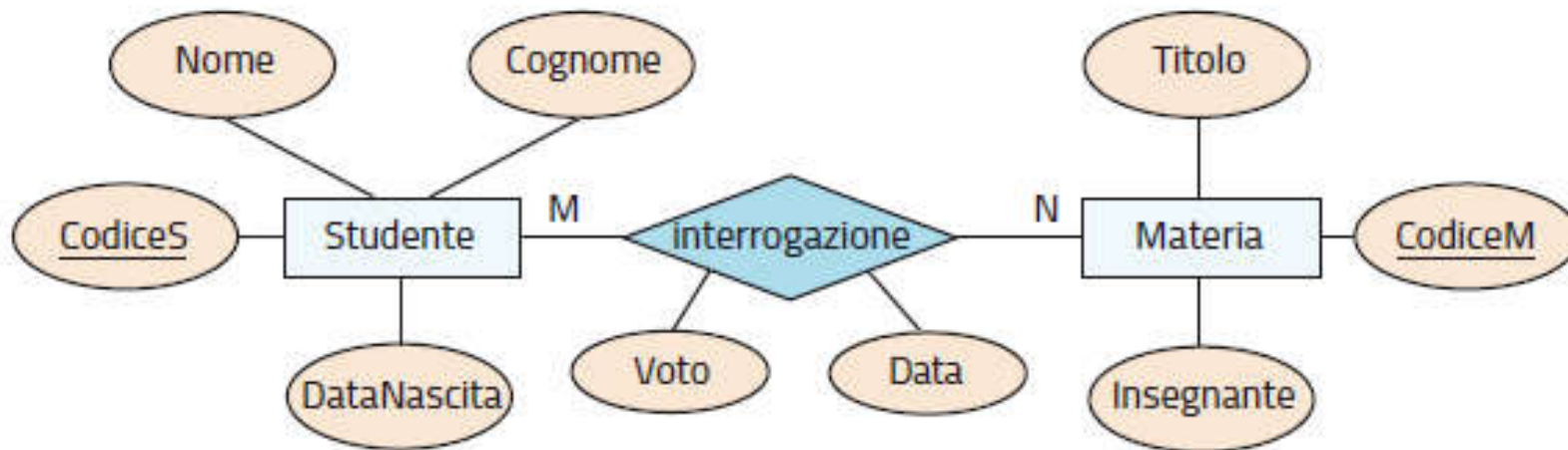
Ogni **associazione** è caratterizzata da una **cardinalità**.



11.3 La progettazione dei database

Indice

Gli **attributi** sono proprietà caratteristiche di un'entità o di un'associazione.
La **chiave primaria** è l'attributo che identifica univocamente le istanze.



gli **attributi** nelle ellissi hanno il nome sottolineato se sono **chiave primaria**

11.4 Il modello logico dei dati e il linguaggio

Indice

Nel **modello relazionale** entità e associazioni sono rappresentate da **tabelle**.

Nel **modello relazionale** la struttura dei dati è descritta da uno **schema** :

STUDENTE (CodiceS, Nome, Cognome, DataNascita, Sesso, ComuneResidenza)

Lo **schema** di una tabella rappresenta la **parte intensionale** del database:

CodiceS	Nome	Cognome	DataNascita	Sesso	ComuneResidenza
S001	Mario	Rossi	08/01/2002	M	Bologna
S002	Giorgia	Verdi	18/09/2001	F	Bologna
S003	Maria	Gialli	01/12/2001	F	Modena
S004	Franco	Neri	07/07/2002	M	Parma

mentre l'**istanza** della tabella (l'insieme dei **record** o **tuple**, le righe di dati) rappresenta la **parte estensionale** del database.

11.4 Il modello logico dei dati e il linguaggio

Indice

Con il **mapping** si può ricavare il modello logico da quello concettuale E-R.

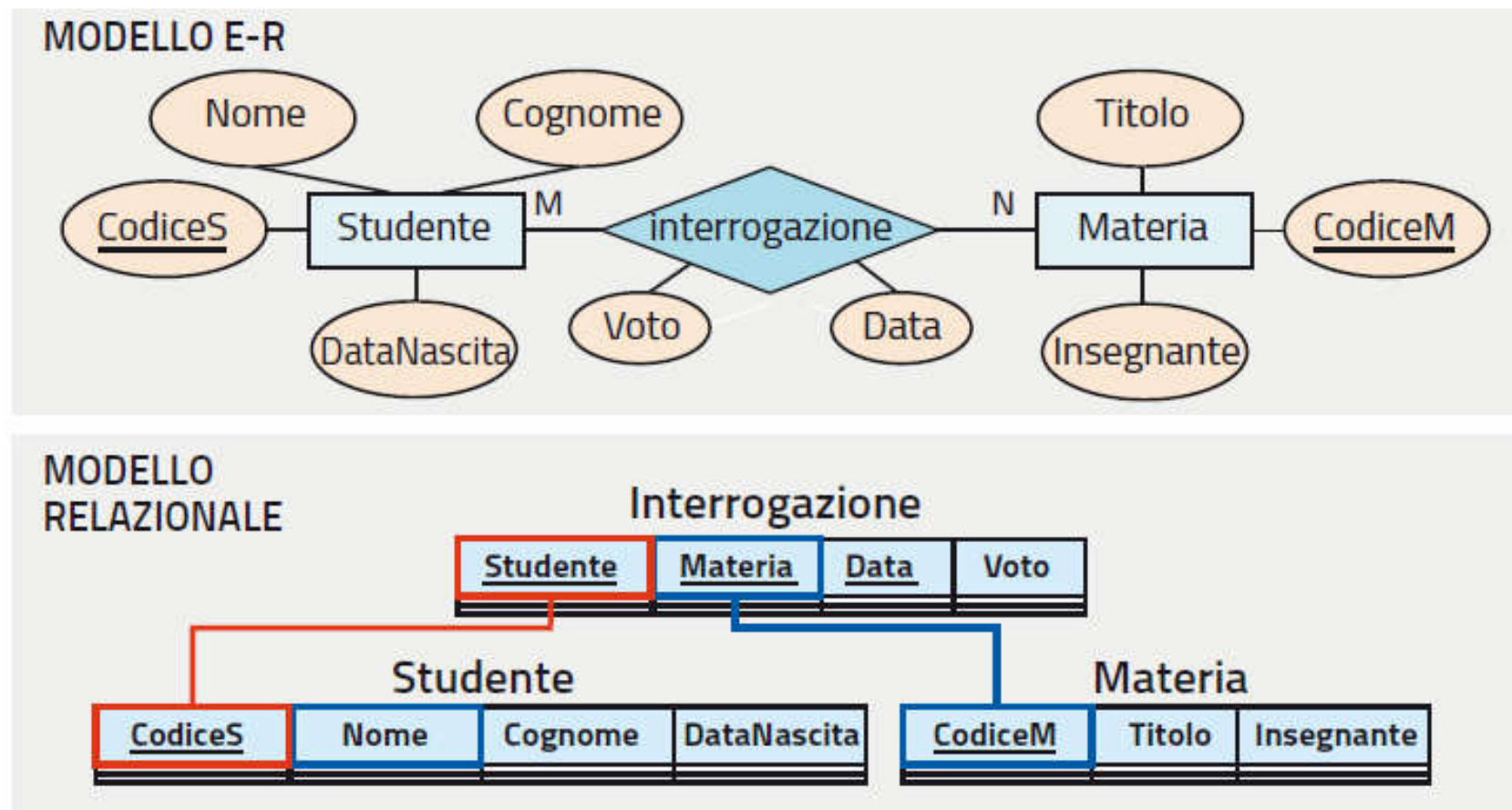
- a ogni **entità** corrisponde lo schema di una **tabella**
- a ogni **attributo** corrisponde un **campo**, cioè l'intestazione di una colonna
- l'**associazione uno-a-molti** si rappresenta assicurandosi che una tabella abbia una **chiave esterna** che assume gli stessi valori della **chiave primaria** dell'altra tabella
- l'**associazione multi-a-molti** si rappresenta tramite **un'ulteriore tabella** con due campi che siano **chiavi esterne** rispetto alle due entità

11.4 Il modello logico dei dati e il linguaggio

Indice

Con il **mapping** si può ricavare il modello logico da quello concettuale E-R.

Un esempio di **mapping** per un'associazione **molti-a-molti**:



11.4 Il modello logico dei dati e il linguaggio

Indice

Il linguaggio **SQL** (***Structured Query Language***) si usa per gestire i dati.

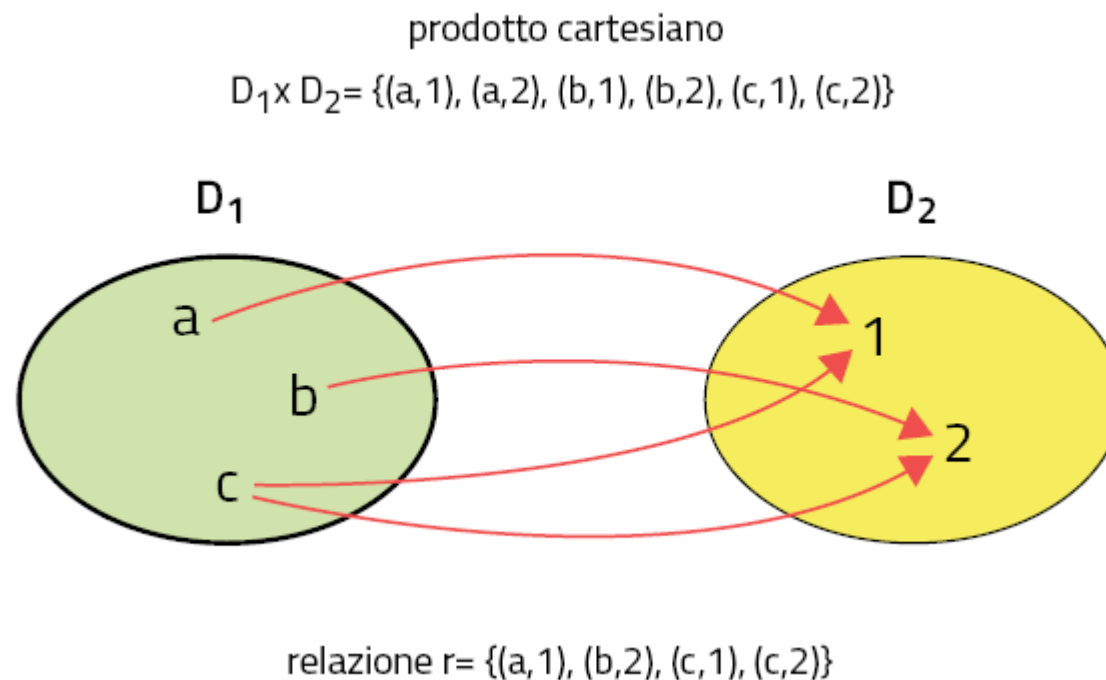
L'**SQL** comprende tre linguaggi specializzati:

- il **DDL** (***Data Definition Language***) per definire gli schemi logici del database
- il **DML** (***Data Manipulation Language***) per interrogare il database e modificarne le istanze
- il **DCL** (***Data Control Language***) per impartire comandi gestionali, come quelli che gestiscono il controllo degli accessi al database

11.5 Le basi teoriche del modello relazionale

Indice

In matematica la **relazione** è una corrispondenza tra elementi di insiemi.

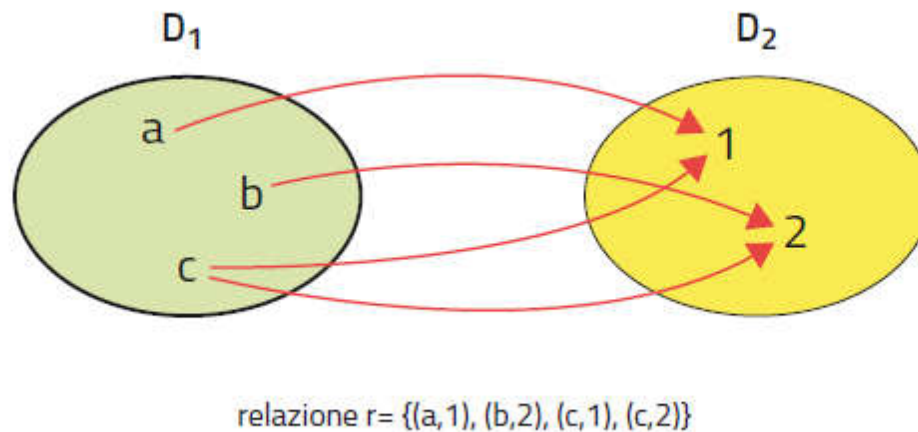


Più precisamente, una **relazione** matematica tra D_1 e D_2 è un **sottoinsieme del prodotto cartesiano** dei due insiemi.

11.5 Le basi teoriche del modello relazionale

Indice

In matematica la **relazione** è una corrispondenza tra elementi di insiemi.



lettere	numeri
a	1
b	2
c	1
c	2

in informatica la relazione si rappresenta con una **tabella**

11.5 Le basi teoriche del modello relazionale

Indice

Così un **database relazionale** è costituito da un certo numero di **tabelle**.

STUDENTE:

CodiceS	Nome	Cognome	DataNascita	Sesso	ComuneResidenza
S001	Mario	Rossi	08/01/2002	M	Bologna
S002	Giorgia	Verdi	18/09/2001	F	Bologna
S003	Maria	Gialli	01/12/2001	F	Modena
S004	Franco	Neri	07/07/2002	M	Parma

MATERIA:

CodiceM	Titolo	Insegnante
M001	Italiano	Bianco
M002	Inglese	Arancio
M003	Matematica	Azzurro

INTERROGAZIONE:

Studente	Materia	Data	Voto
S001	M003	20/04/2020	8.0
S001	M001	10/05/2020	6.5
S004	M003	20/04/2020	5.5
S001	M002	02/06/2020	9.0
S003	M003	02/06/2020	8.5
S004	M001	20/03/2020	4.0
S002	M003	20/04/2020	7.5

le relazioni devono
preferibilmente essere in **1NF**

11.6 I valori nulli e i vincoli sui dati

Indice

I **valori nulli** rappresentano dati mancanti nelle istanze delle tabelle.

Il **valore NULL** è un valore speciale che non appartiene ad alcun dominio.

In particolare, se due record hanno entrambi valore **NULL** per un certo campo, *non se ne può dedurre* che abbiano per quell'attributo lo stesso valore.

11.6 I valori nulli e i vincoli sui dati

Indice

I **vincoli** sono le condizioni che i dati devono rispettare.

Esempi:

vincolo di dominio: $(Voto \geq 0) \text{ AND } (Voto \leq 10)$ per il campo `Voto`

vincolo di tupla: `ImportoLordo = ImportoNetto + IVA` per una tabella con schema `PAGAMENTO (Data, ImportoLordo, IVA, ImportoNetto)`

vincolo di chiave: vieta che due tuple possano avere lo stesso valore di chiave

Nel modello relazionale ogni relazione deve avere un campo **chiave primaria** che *non ammetta* valori NULL.

11.6 I valori nulli e i vincoli sui dati

Indice

I **vincoli di integrità referenziale** assicurano la validità delle chiavi esterne.

STUDENTE:

<u>CodiceS</u>	Nome	Cognome	DataNascita	Sesso	ComuneResidenza
S001	Mario	Rossi	08/01/2002	M	Bologna
S002	Giorgia	Verdi	18/09/2001	F	Bologna
S003	Maria	Gialli	01/12/2001	F	Modena
S004	Franco	Neri	07/07/2002	M	Parma

MATERIA:

<u>CodiceM</u>	Titolo	Insegnante
M001	Italiano	Bianco
M002	Inglese	Arancio
M003	Matematica	Azzurro

INTERROGAZIONE:

<u>Studente</u>	<u>Materia</u>	<u>Data</u>	<u>Voto</u>
S001	M003	20/04/2020	8.0
S001	M001	10/05/2020	6.5
S004	M003	20/04/2020	5.5
S001	M002	02/06/2020	9.0
S003	M003	02/06/2020	8.5
S004	M001	20/03/2020	4.0
S002	M003	20/04/2020	7.5

I **vincoli di integrità referenziale** sono fondamentali: permettono di correlare tra loro i dati di diverse tabelle mediante l'uso di **chiavi esterne** o **FK**.

capitolo 12 – SQL: la definizione dei dati e le query

1. Definire gli schemi delle tabelle
2. Definire i vincoli e modificare le tabelle
3. Popolare le tabelle e fare query semplici
4. Usare gli operatori e ordinare i risultati
5. Fare query su più tabelle: le clausole **JOIN**

12.1 Definire gli schemi delle tabelle

Indice

In SQL si possono **creare le tabelle** di un db specificando schema e vincoli.

Per **creare un database** si scrive l'istruzione SQL:

```
CREATE DATABASE nome_db;
```

Per **creare una nuova tabella** nel db:

```
CREATE TABLE nome_tabella  
(  
  nome_attr1 tipo1 vincolo1,  
  nome_attr2 tipo2 vincolo2,  
  ... ,  
  nome_attrN tipoN vincoloN,  
  vincolo_di_tabella  
);
```

Il **tipo di dato** degli attributi può essere:

- stringa di caratteri
- valore numerico
- data e ora
- valore booleano
- dato in codice binario

12.1 Definire gli schemi delle tabelle

Indice

In SQL si possono **creare le tabelle** di un db specificando schema e vincoli.

Esempio: creare il database **Classe** e lo schema della tabella **Studente**.

```
CREATE DATABASE Classe;
```

```
CREATE TABLE Studente (  
  CodiceS char(4),  
  Nome varchar(30),  
  Cognome varchar(50),  
  DataNascita date,  
  Sesso char(1),  
  ComuneResidenza varchar(60),  
);
```

CodiceS	Nome	Cognome	DataNascita	Sesso	ComuneResidenza

12.1 Definire gli schemi delle tabelle

Indice

In SQL si possono **creare le tabelle** di un db specificando schema e vincoli.

I principali **tipi di dato**
nel DBMS **MySQL**:

TIPO DI DATO	DESCRIZIONE
stringhe	
CHAR (<i>d</i>)	contiene una stringa di lunghezza fissa <i>d</i> (con default: <i>d</i> =1); se si inserisce una stringa più corta, il DBMS aggiunge in coda spazi bianchi fino al raggiungimento della lunghezza dichiarata <i>d</i>
VARCHAR (<i>d</i>)	contiene una stringa di lunghezza variabile, fino a un numero massimo di caratteri alfanumerici pari a <i>d</i>
TEXT	contiene una stringa con una lunghezza massima di 65 535 caratteri
LONGTEXT	contiene una stringa con lunghezza fino a 4 gigabyte
numeri	
TINYINT (<i>n</i>)	contiene un numero intero compreso tra -128 e 127 (o tra 0 e 255 se UNSIGNED); l'indicazione del numero massimo di cifre <i>n</i> è opzionale
INT (<i>n</i>)	contiene un intero compreso tra -2 147 483 648 e 2 147 483 647 (o tra 0 e 4 294 967 295 se UNSIGNED); l'indicazione del numero massimo di cifre <i>n</i> è opzionale
FLOAT (<i>n,d</i>)	contiene un numero decimale a virgola mobile in singola precisione, con <i>d</i> cifre decimali; il numero massimo di cifre totali <i>n</i> è opzionale
DOUBLE (<i>n,d</i>)	contiene un numero decimale come sopra, ma in doppia precisione
DECIMAL (<i>n,d</i>)	contiene numeri decimali in doppia precisione memorizzati come stringhe; permette di registrare valori esatti in virgola fissa, così da evitare i problemi di arrotondamento possibili con FLOAT e DOUBLE
date e ore	
DATE ()	contiene una data nel formato yyyy-mm-dd
TIME ()	contiene un orario nel formato HH:MM:SS
DATETIME ()	contiene una combinazione di data e ora yyyy-mm-dd HH:MM:SS
booleano	
BOOL	può assumere soltanto i due valori vero (TRUE) o falso (FALSE)
dati binari	
BLOB	contiene bit 0 e 1 fino a una lunghezza massima di 65 535 byte
LOB	contiene bit 0 e 1 fino a una lunghezza massima di 4 GB

12.2 Definire i vincoli e modificare le tabelle

Indice

I **vincoli** si possono specificare all'atto della creazione delle tabelle.

Esempi:

`password char(12) NOT NULL,`

vieta la presenza di valori nulli nell'attributo `password`

`Professione char(30) DEFAULT 'Programmatore',`

se non si indica un valore, il campo `Professione` conterrà `'Programmatore'`

`stipendio float CHECK (stipendio >0),`

il DBMS controllerà che il campo `stipendio` contenga soltanto valori positivi

12.2 Definire i vincoli e modificare le tabelle

Indice

Anche le **chiavi** vanno specificate all'atto della creazione delle tabelle.

il **vincolo di chiave** si può scrivere in linea, nell'elenco degli attributi:

`CodiceISBN char(17) PRIMARY KEY,`

oppure come vincolo di tabella, dopo aver dichiarato tutti gli attributi:

`PRIMARY KEY (Titolo,Autore)`

12.2 Definire i vincoli e modificare le tabelle

Indice

Anche le **chiavi** vanno specificate all'atto della creazione delle tabelle.

Esempio: definizione della tabella `Studente` con vincoli e chiavi:

```
CREATE TABLE Studente (  
  CodiceS      char(4)          PRIMARY KEY, -- chiave primaria  
  Nome         varchar(30)      NOT NULL,  
  Cognome      varchar(50)      NOT NULL,  
  DataNasc     date             CHECK (DataNasc < 2000-01-01)  
  Sesso        char(1),  
  ComuneResidenza varchar(60)  DEFAULT 'Bologna',  
  UNIQUE (Nome, Cognome) -- chiave  
)
```

12.2 Definire i vincoli e modificare le tabelle

Indice

Per definire le **chiavi esterne** in SQL si usa la clausola **REFERENCES**.

Anche qui il **vincolo di chiave** si può scrivere in linea, nell'elenco degli attributi:

```
Studente char(4) REFERENCES STUDENTE(CodiceS) ,
```

oppure come vincolo di tabella, dopo aver dichiarato tutti gli attributi:

```
FOREIGN KEY (Studente) REFERENCES STUDENTE(CodiceS)
```

Esempio di tabella con 2 chiavi esterne:

```
CREATE TABLE Interrogazione (  
  Studente char(4) REFERENCES STUDENTE(CodiceS) , -- FK  
  Materia char(4) REFERENCES MATERIA(CodiceM) , -- FK  
  Data date,  
  Voto float CHECK (Voto >= 0 AND Voto < 10) ,  
  PRIMARY KEY (Studente,Materia,Data)  
)
```

12.2 Definire i vincoli e modificare le tabelle

Indice

L'istruzione SQL **ALTER TABLE** modifica lo schema di una tabella.

Esempio: un'istruzione che aggiunge (**ADD**) un campo, elimina (**DROP**) un vincolo esistente e aggiunge un vincolo nuovo

ALTER TABLE Interrogazione

ADD ScrittOrale char(1) CHECK (ScrittOrale in ('S','O'))

DROP CONSTRAINT CHECK (Voto >= 0.0 AND Voto <= 10.0)

ADD CONSTRAINT CHECK (Voto >= 3.0 AND Voto <= 10.5);

12.3 Popolare le tabelle e fare query semplici

Indice

Il linguaggio **DML** permette di gestire i record delle tabelle.

Esempio: istruzione che aggiunge un record alla tabella **STUDENTE**

```
INSERT INTO STUDENTE (  
CodiceS , Nome , Cognome , DataNascita , Sesso , ComuneResidenza )  
VALUES ( 'S001' , 'Mario' , 'Rossi' , '2002-01-08' , 'M' , 'Bologna' ) ;
```

CodiceS	Nome	Cognome	DataNascita	Sesso	ComuneResidenza
S001	Mario	Rossi	2002-01-08	M	Bologna

12.3 Popolare le tabelle e fare query semplici

Indice

L'istruzione **SELECT** permette di scrivere *query*, cioè interrogazioni sui dati.

```
SELECT attributi della tabella  
FROM nome_tabella  
WHERE condizione da soddisfare ;
```

Esempio:

```
SELECT Nome, Cognome  
FROM Studente  
WHERE Sesso='F' AND ComuneResidenza='Bologna' ;
```

CodiceS	Nome	Cognome	DataNascita	Sesso	ComuneResidenza
S001	Mario	Rossi	2002-01-08	M	Bologna
S002	Giorgia	Verdi	2001-09-18	F	Bologna
S003	Maria	Gialli	2001-12-01	F	Modena
S004	Franco	Neri	2002-07-07	M	Parma

Risultato:

Nome	Cognome
Giorgia	Verdi

Se si scrive **SELECT * FROM ...**, verranno selezionati **tutti i campi** della tabella.

12.3 Popolare le tabelle e fare query semplici

Indice

In SQL si possono assegnare **alias** per scrivere istruzioni più concise.

Esempio: scrivendo l'istruzione

```
Select S.Nome , S.Cognome from Studente S,
```

si può fare riferimento alla tabella **Studente** chiamandola semplicemente S

12.3 Popolare le tabelle e fare query semplici

Indice

Si possono **inserire in una tabella più record** con un'unica istruzione, usando i dati restituiti come risultato di una **query**.

Esempio: con l'istruzione

```
INSERT INTO StudentiBologna (CodiceS, Nome, Cognome)
SELECT CodiceS, Nome, Cognome
FROM Studente
WHERE ComuneResidenza = 'Bologna';
```

si aggiungeranno alla tabella `StudentiBologna` i dati di tutti i record della tabella `Studenti` che corrispondono a studenti residenti a Bologna

12.3 Popolare le tabelle e fare query semplici

Indice

Con **UPDATE** si possono modificare i record, con **DELETE** li si cancella.

Esempio: l'istruzione SQL

UPDATE *Studente*

SET *ComuneResidenza* = 'Milano'

WHERE *ComuneResidenza* = 'Bologna';

fa diventare residenti a Milano tutti gli studenti residenti a Bologna

(se la clausola **WHERE** non viene specificata, verranno modificati tutti i record)

Altro esempio: l'istruzione

DELETE **FROM** *Studente* **WHERE** *ComuneResidenza* = 'Bologna';

elimina dalla tabella *Studente* i record relativi ai residenti a Bologna

12.4 Usare gli operatori e ordinare i risultati

Indice

Gli **operatori** dell'SQL tornano utili nella **condizione della clausola WHERE**.

Gli **operatori di confronto tra valori** sono:

uguale a	=
maggiore di	>
minore di	<
maggiore o uguale a	>=
minore o uguale a	<=
diverso da	<>

12.4 Usare gli operatori e ordinare i risultati

Indice

Gli **operatori** dell'SQL tornano utili nella **condizione della clausola WHERE**.

L'**operatore LIKE** permette di fare confronti tra stringhe usando le **wildcard** o **caratteri jolly**: **_** rappresenta un qualsiasi carattere, **%** una qualsiasi stringa.

Esempio:

```
SELECT CodiceS, Nome, Cognome FROM Studente  
WHERE Nome LIKE 'M%';
```

restituisce i dati relativi agli studenti il cui nome inizia per **M**

Altro esempio:

```
SELECT CodiceS, Nome, Cognome FROM Studente  
WHERE Nome LIKE '_i%a';
```

restituisce i dati relativi agli studenti il cui nome ha **i** come seconda lettera e termina con **a**

12.4 Usare gli operatori e ordinare i risultati

Indice

Gli **operatori** dell'SQL tornano utili nella **condizione della clausola WHERE**.

Con l'**operatore BETWEEN** si verifica l'appartenenza a un intervallo.

INTERROGAZIONE

Studente	Materia	Data	Voto
S001	M003	2020-04-20	8.0
S001	M001	2020-05-10	6.5
S004	M003	2020-04-20	5.5
S001	M002	2020-06-02	9.0
S003	M003	2020-06-02	8.5
S004	M001	2020-03-20	4.0
S002	M003	2020-04-20	7.5

SELECT Materia,Voto **FROM** Interrogazione
WHERE Voto **BETWEEN** 8 and 10;

Risultato:

Materia	Voto
M003	8.0
M002	9.0
M003	8.5

12.4 Usare gli operatori e ordinare i risultati

Indice

Gli **operatori** dell'SQL tornano utili nella **condizione della clausola WHERE**.

Con l'**operatore IN** si verifica l'appartenenza a un insieme specificato.

INTERROGAZIONE

Studente	Materia	Data	Voto
S001	M003	2020-04-20	8.0
S001	M001	2020-05-10	6.5
S004	M003	2020-04-20	5.5
S001	M002	2020-06-02	9.0
S003	M003	2020-06-02	8.5
S004	M001	2020-03-20	4.0
S002	M003	2020-04-20	7.5

```
SELECT Materia,Voto FROM Interrogazione  
WHERE Materia IN ('M001','M003');
```

Risultato:

Studente
S001
S004

12.4 Usare gli operatori e ordinare i risultati

Indice

Gli **operatori logici** permettono di **combinare tra loro più condizioni**.

AND	V	F
V	V	F
F	F	F

OR	V	F
V	V	V
F	V	F

NOT	
V	F
F	V

```
SELECT CodiceS, Nome, Cognome, ComuneResidenza FROM Studente  
WHERE Sesso='F' AND ComuneResidenza='Bologna';
```

cerca gli studenti che sono *sia* di sesso femminile, *sia* residenti a Bologna

```
SELECT CodiceS, Nome, Cognome, ComuneResidenza FROM Studente  
WHERE Sesso='F' OR ComuneResidenza='Bologna';
```

meno restrittiva: cerca gli studenti di sesso femminile *oppure* residenti a Bologna

12.4 Usare gli operatori e ordinare i risultati

Indice

Per **ordinare i risultati di una query** si può usare la clausola **ORDER BY**.

MATERIA	CodiceM	Titolo	Insegnante
	M001	Italiano	Bianco
	M002	Inglese	Arancio
	M003	Matematica	Azzurro

```
SELECT * FROM Materia  
ORDER BY Insegnante;
```

Risultato:

CodiceM	Titolo	Insegnante
M002	Inglese	Arancio
M003	Matematica	Azzurro
M001	Italiano	Bianco

```
SELECT * FROM Materia  
ORDER BY Insegnante  
DESC;
```

Risultato:

CodiceM	Titolo	Insegnante
M001	Italiano	Bianco
M003	Matematica	Azzurro
M002	Inglese	Arancio

12.5 Fare query su più tabelle: le clausole JOIN

Indice

Le **clausole JOIN** consentono di **collegare dati presenti in tabelle diverse**.

MATERIA

CodiceM	Titolo	Insegnante
M001	Italiano	Bianco
M002	Inglese	Arancio
M003	Matematica	Azzurro

query complessa: cerchiamo le interrogazioni con **voto** tra 8 e 10 e con la **materia** indicata per esteso

INTERROGAZIONE

Studente	Materia	Data	Voto
S001	M003	2020-04-20	8.0
S001	M001	2020-05-10	6.5
S004	M003	2020-04-20	5.5
S001	M002	2020-06-02	9.0
S003	M003	2020-06-02	8.5
S004	M001	2020-03-20	4.0
S002	M003	2020-04-20	7.5

```
SELECT M.CodiceM, M.Titolo, M.Insegnante, I.Voto
FROM Materia M JOIN Interrogazione I ON M.CodiceM = I.Materia
WHERE I.Voto BETWEEN 8. AND 10.;
```

NB: deve interrogare *simultaneamente* le tabelle MATERIA e INTERROGAZIONE.

12.5 Fare query su più tabelle: le clausole JOIN

Indice

Le **clausole JOIN** consentono di **collegare dati presenti in tabelle diverse**.

```
SELECT M.CodiceM, M.Titolo, M.Insegnante, I.Voto  
FROM Materia M JOIN Interrogazione I ON M.CodiceM = I.Materia  
WHERE I.Voto BETWEEN 8. AND 10.;
```

1. il DBMS, in base alla clausola **FROM**, costruisce questa tabella con righe formate dall'unione di record delle due tabelle MATERIA e INTERROGAZIONE

M.CodiceM	M.Titolo	M.Insegnante	M.Studente	I.Materia	I.Data	I.Voto
M001	Italiano	Bianco	S001	M001	2020-05-10	6.5
M001	Italiano	Bianco	S004	M001	2020-03-20	4.0
M002	Inglese	Arancio	S001	M002	2020-06-02	9.0
M003	Matematica	Azzurro	S001	M003	2020-01-20	8.0
M003	Matematica	Azzurro	S004	M003	2020-04-20	5.5
M003	Matematica	Azzurro	S003	M003	2020-06-02	8.5
M003	Matematica	Azzurro	S002	M003	2020-04-20	7.5

12.5 Fare query su più tabelle: le clausole JOIN

Indice

Le **clausole JOIN** consentono di **collegare dati presenti in tabelle diverse**.

```
SELECT M.CodiceM, M.Titolo, M.Insegnante, I.Voto
FROM Materia M JOIN Interrogazione I ON M.CodiceM = I.Materia
WHERE I.Voto BETWEEN 8. AND 10.;
```

1. il DBMS, in base alla clausola **FROM**, costruisce la seguente tabella, con righe formate dall'unione di record delle due tabelle MATERIA e INTERROGAZIONE:

M.CodiceM	M.Titolo	M.Insegnante	M.Studente	I.Materia	I.Data	I.Voto
M001	Italiano	Bianco	S001	M001	2020-05-10	6.5
M001	Italiano	Bianco	S004	M001	2020-03-20	4.0
M002	Inglese	Arancio	S001	M002	2020-06-02	9.0
M003	Matematica	Azzurro	S001	M003	2020-01-20	8.0
M003	Matematica	Azzurro	S004	M003	2020-04-20	5.5
M003	Matematica	Azzurro	S003	M003	2020-06-02	8.5
M003	Matematica	Azzurro	S002	M003	2020-04-20	7.5

2. poi il DBMS applica alla tabella la clausola **WHERE**, identificando così **3 record**

3. infine estrae dalla tabella, per quei record, gli **attributi elencati in SELECT**

12.5 Fare query su più tabelle: le clausole JOIN

Indice

Le **clausole JOIN** consentono di **collegare dati presenti in tabelle diverse**.

Dunque nelle **query complesse** l'operazione di (**INNER**) **JOIN** tra due tabelle:

1. crea una nuova tabella combinando i valori delle due tabelle di input per tutte quelle righe che soddisfano la regola di confronto del predicato di **JOIN**;
2. poi seleziona le righe che soddisfano la condizione della clausola **WHERE**;
3. infine estrae le colonne richieste dalla clausola **SELECT**.

La sintassi generale è:

```
SELECT elenco di attributi per l'output
FROM nome_tabella1 AS tab1
JOIN nome_tabella2 AS tab2 ON tab1.attribX = tab2.attribY
WHERE condizione da soddisfare;
```

Di solito nel **predicato di JOIN** (la sotto-clausola **ON**) **attribX** e **attribY** sono la **primary key** di una tabella e una **foreign key** dell'altra tabella.

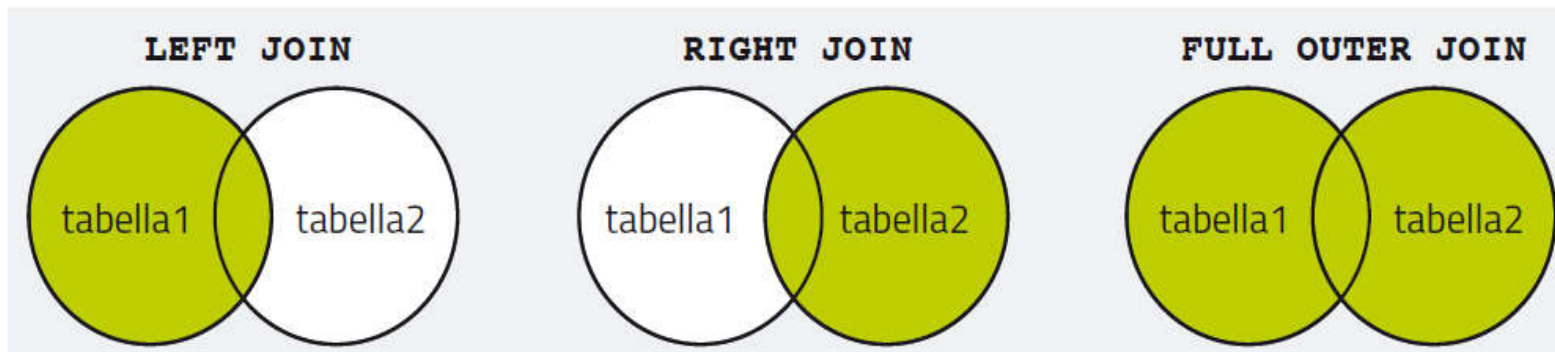
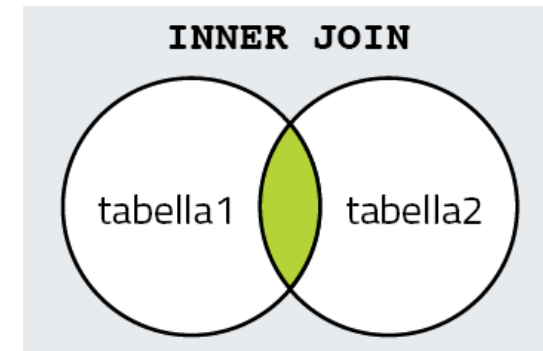
12.5 Fare query su più tabelle: le clausole JOIN

Indice

Le **clausole JOIN** consentono di **collegare dati presenti in tabelle diverse**.

L'operazione di (**INNER**) JOIN richiede che vi sia corrispondenza esatta tra le righe delle due tabelle.

Questo vincolo *non* si applica all'**OUTER JOIN** che trattiene, oltre ai record con valori coincidenti degli attributi della regola di JOIN, anche i **record *dangling***, privi di corrispondenza nelle due tabelle.



Si può avere **LEFT (OUTER) JOIN**, **RIGHT (OUTER) JOIN** oppure **FULL (OUTER) JOIN**, a seconda della tabella di cui si trattengono i record in caso di mancata corrispondenza della regola di JOIN.

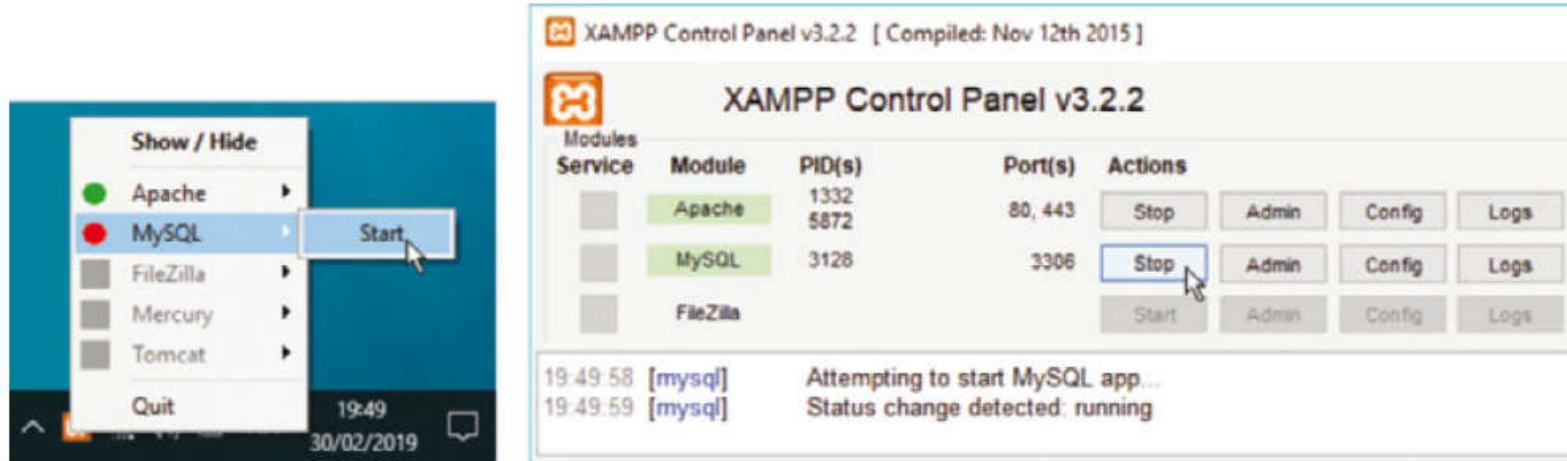
capitolo 13 – I database online con MySql e PHP

1. Creare un database online
2. Creare le tabelle
3. Inserire i dati e modificarli
4. Eseguire le query

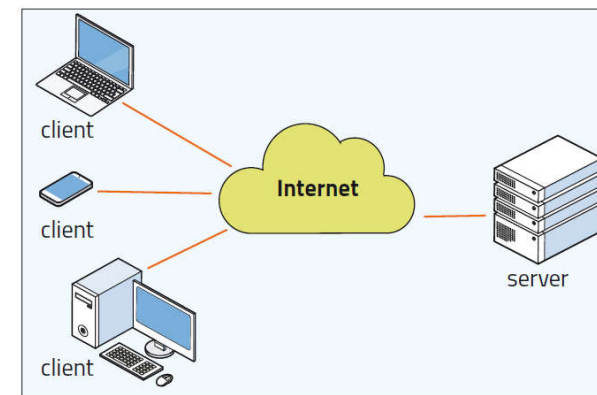
13.1 Creare un database online

Indice

MySQL è un DBMS open source molto diffuso, incluso nella suite **XAMPP**.



Per gestire un **database su web server** si può usare un DBMS come **MySQL** combinato con i linguaggi SQL, PHP, HTML e CSS.

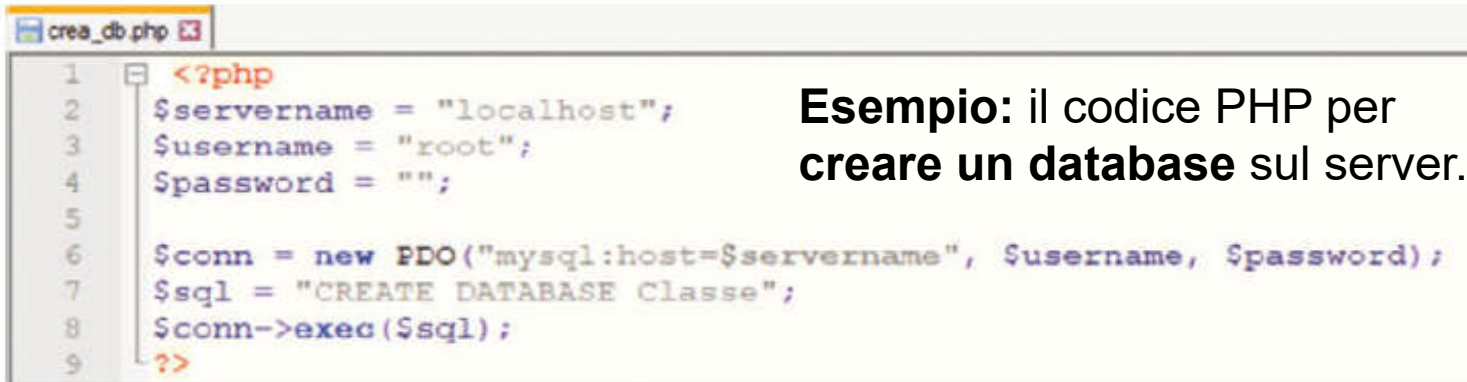


13.1 Creare un database online

Indice

Il linguaggio **PHP** permette di fare molte operazioni sui database.

- creare un database sul web server
- inserire dati o modificare i dati già presenti
- fare ricerche tra i dati
- eliminare dati



Esempio: il codice PHP per creare un database sul server.

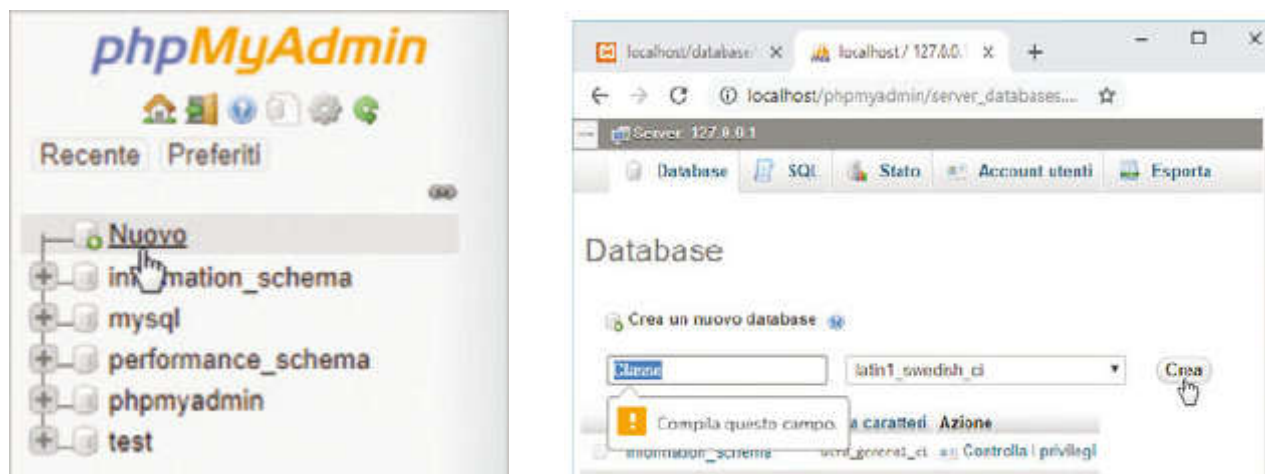
```
1 <?php
2 $servername = "localhost";
3 $username = "root";
4 $password = "";
5
6 $conn = new PDO("mysql:host=$servername", $username, $password);
7 $sql = "CREATE DATABASE Classe";
8 $conn->exec($sql);
9 ?>
```

Il **PDO** (estensione del PHP) consente di eseguire comandi SQL: si stabilisce una connessione, si imposta una stringa con i comandi da eseguire e la funzione **exec** li esegue.

13.1 Creare un database online

Indice

phpMyAdmin offre una comoda **interfaccia grafica** per l'utente di MySQL.



phpMyAdmin si apre al clic sul pulsante **Admin** posto a fianco di **MySQL** nel pannello di controllo di XAMPP.

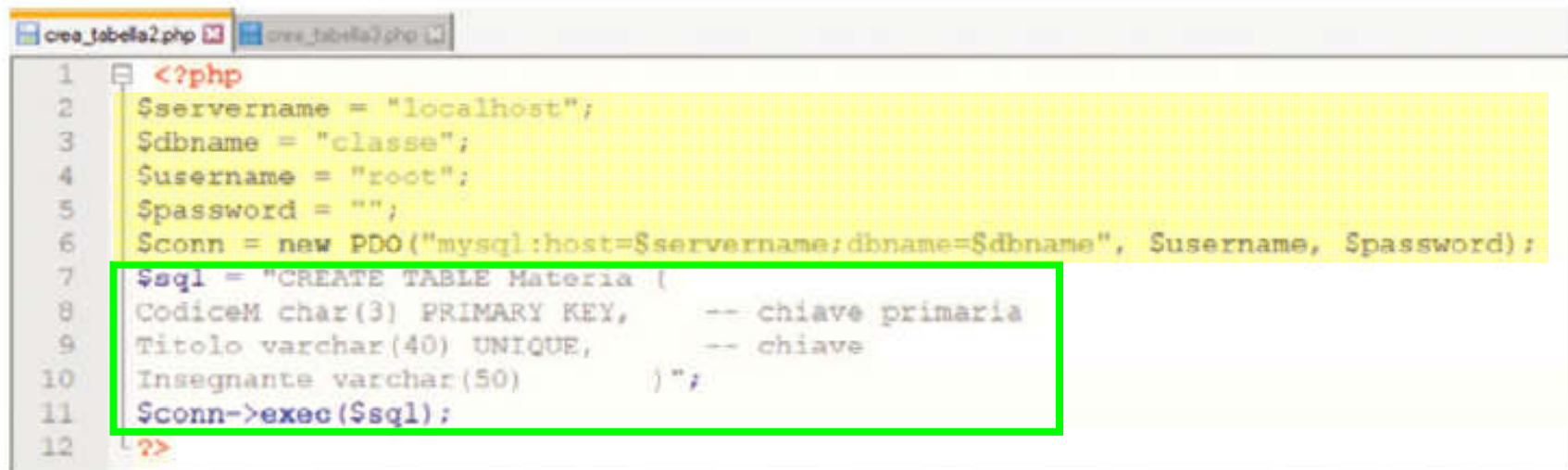
Con questa **interfaccia grafica** l'utente remoto può interagire comodamente con i database sul server.

Esempio: per creare un nuovo database, basta fare clic su **Nuovo** nel riquadro di sinistra, poi digitare il nome del nuovo database e fare clic su **Crea**.

13.2 Creare le tabelle

Indice

Le **tabelle** si possono creare facendo eseguire al server un codice scritto in PHP e PDO.



```
1 <?php
2 $servername = "localhost";
3 $dbname = "classe";
4 $username = "root";
5 $password = "";
6 $conn = new PDO("mysql:host=$servername;dbname=$dbname", $username, $password);
7 $sql = "CREATE TABLE Materia (
8     CodiceM char(3) PRIMARY KEY,      -- chiave primaria
9     Titolo varchar(40) UNIQUE,        -- chiave
10    Insegnante varchar(50)             );";
11 $conn->exec($sql);
12
```

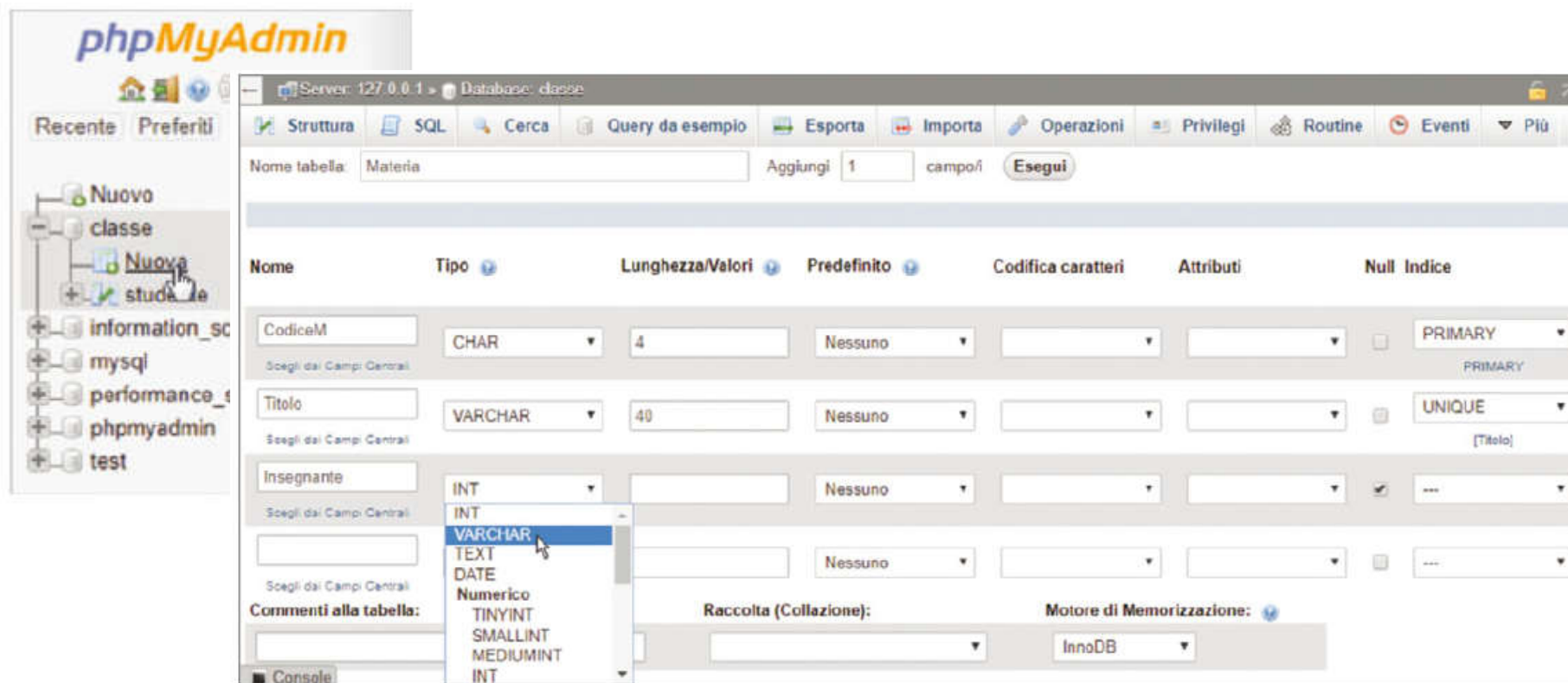
Esempio: per creare la tabella MATERIA, si assegna l'istruzione SQL alla variabile stringa `$sql`, che è poi passata a `exec` per l'esecuzione.

Nota: ogni volta che uno script viene eseguito, la connessione al database viene chiusa automaticamente. Perciò ogni nuovo script che si vuole eseguire deve contenere i parametri per la connessione.

13.2 Creare le tabelle

Indice

Le **tabelle** si possono creare anche con l'interfaccia grafica di **phpMyAdmin**.



Basta scegliere **Nuova** nella struttura ad albero e poi usare le caselle e i menu.

In seguito con la stessa interfaccia si potrà **modificare lo schema** della tabella.

13.3 Inserire i dati e modificarli

Indice

Per **inserire dati nelle tabelle** si può eseguire un codice in PHP e PDO.



```
1 <?php
2 $server = "localhost";
3 $db = "classe";
4 $username = "root";
5 $password = "";
6
7 $conn = new PDO("mysql:host=$server;dbname=$db", $username, $password);
8 $sql = "INSERT INTO STUDENTE
9 VALUES('S002', 'Giorgia', 'Verdi', '2001-09-18', 'F', 'Bologna')";
10 $conn->exec($sql);
11 ?>
```

Esempio: il codice che inserisce un record nella tabella **STUDENTE**.

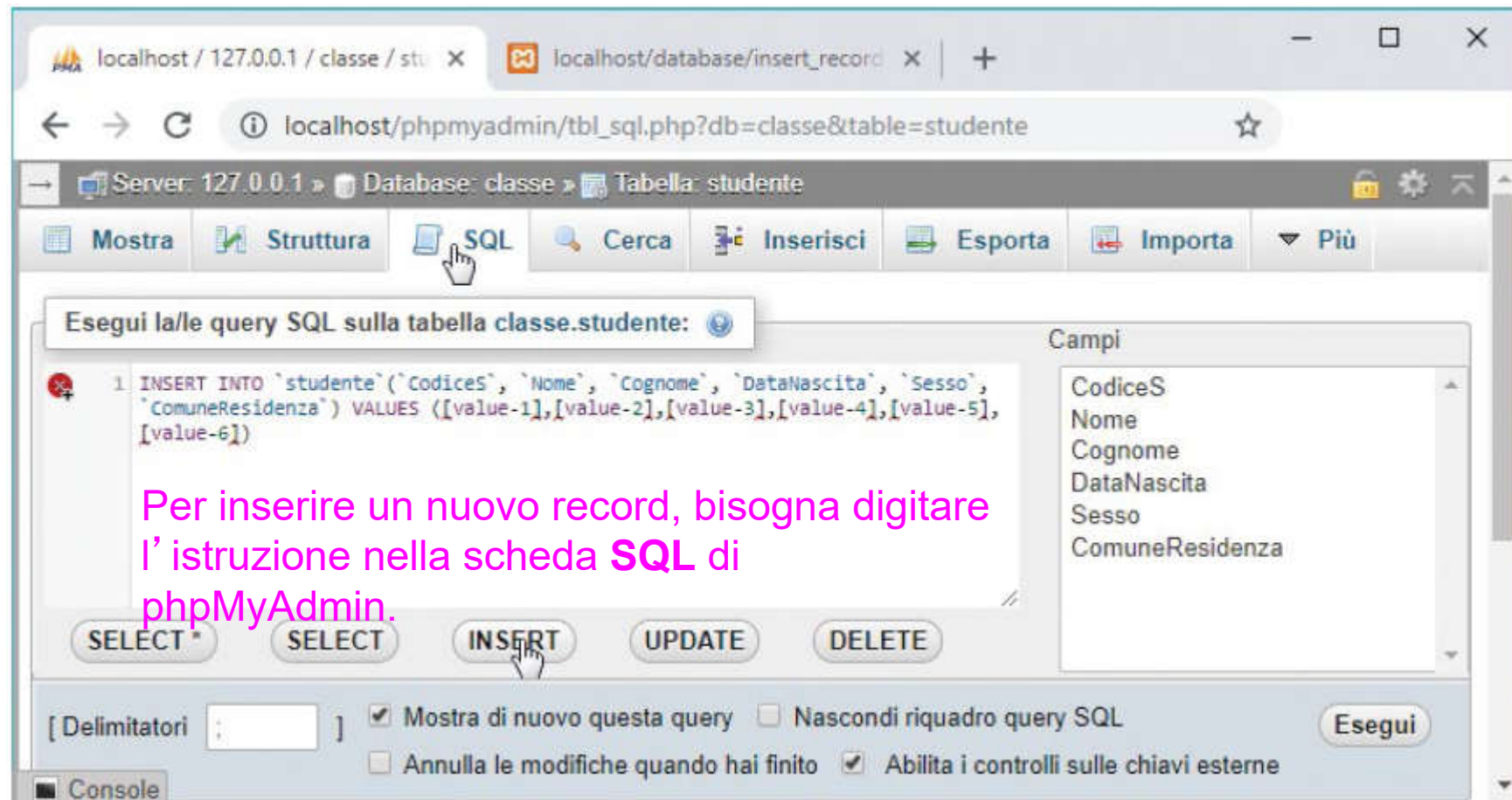
Una volta eseguito l'inserimento, l'interfaccia grafica di phpMyAdmin consente di verificare l'effettiva variazione del contenuto del database.

Con la stessa tecnica si possono eseguire anche operazioni **UPDATE** per modificare i record già esistenti.

13.3 Inserire i dati e modificarli

Indice

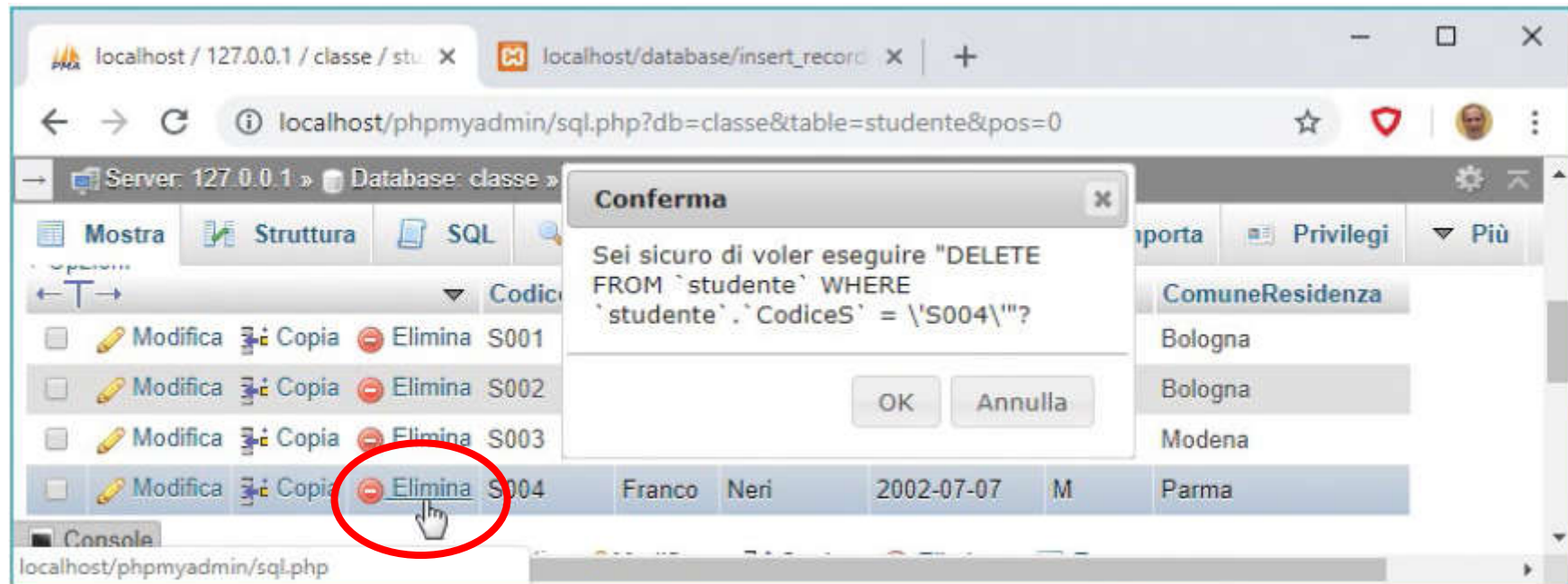
L'**inserimento dei dati** è laborioso anche se si usa l'interfaccia grafica.



13.3 Inserire i dati e modificarli

Indice

L'interfaccia grafica facilita enormemente la **modifica dei dati esistenti**.



Esempio: per **eliminare un record** basta un clic su un pulsante.

13.4 Eseguire le query

Indice

L'esecuzione delle **query** richiede di gestire i dati ricevuti come risposta.

```
1 <?php
2 $server = "localhost";
3 $db = "classe";
4 $username = "root";
5 $password = "";
6
7 $conn = new PDO("mysql:host=$server;dbname=$db", $username, $password);
8 foreach ($conn->query("SELECT CodiceS, Nome, Cognome, ComuneResidenza
9                      FROM Studente WHERE Sesso='F' OR ComuneResidenza='Bologna'") as $risq)
10 {echo "<br>".$risq['CodiceS']." ".$risq['Nome']." ".$risq['Cognome']." ".$risq['ComuneResidenza'];}
11 ?>
```

la funzione **query** dice al DBMS che la stringa tra parentesi è un'istruzione SQL che specifica l'interrogazione



localhost/database/query_esemp x +
localhost/database/query_esempio1.php ☆

S001	Mario	Rossi	Bologna
S002	Giorgia	Verdi	Bologna
S003	Maria	Gialli	Modena

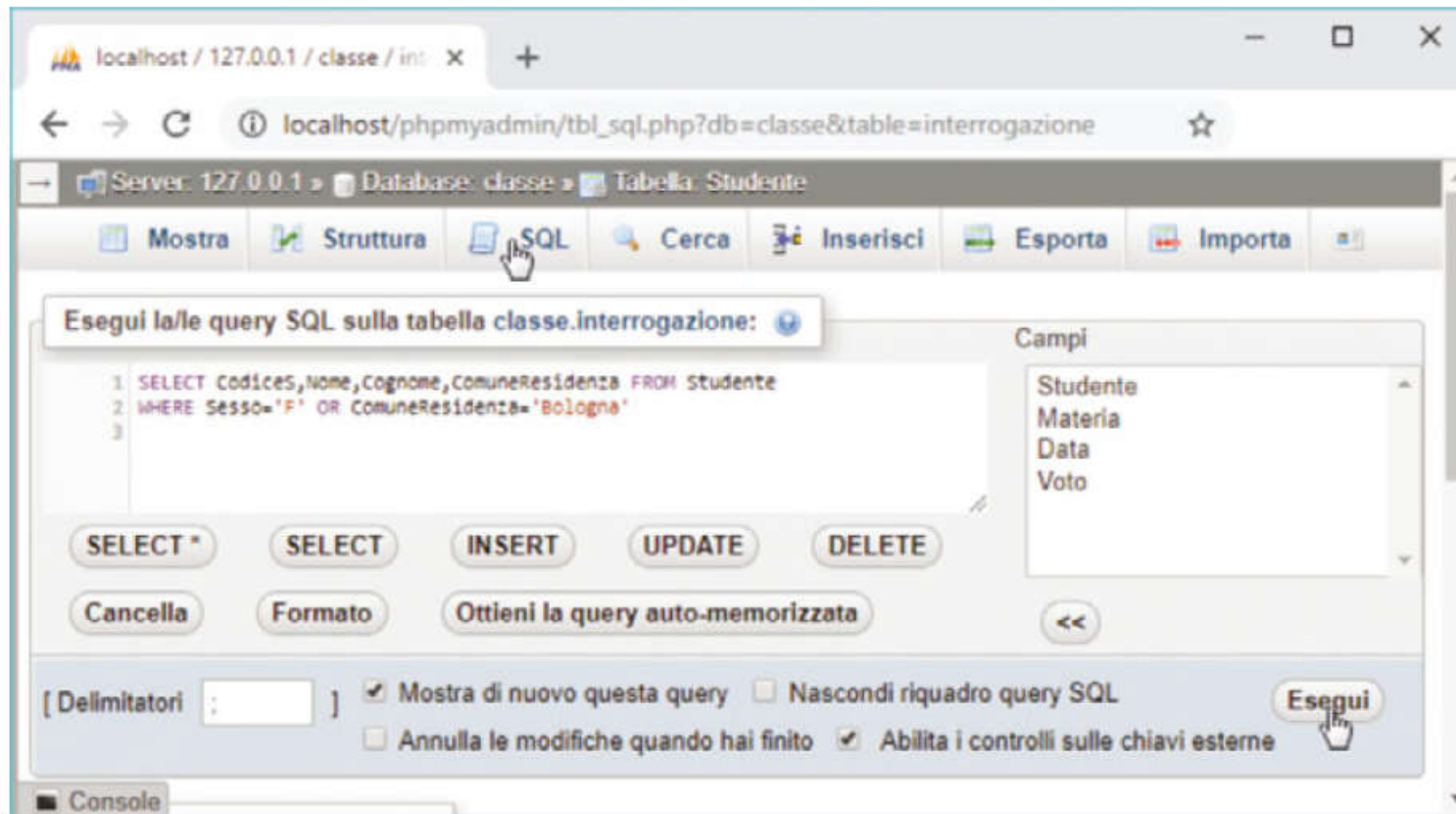
i risultati della query sono raccolti nell'array dinamico **\$risq**, che ha come indici i nomi delle colonne

il comando PHP **echo** visualizza sulla pagina web i risultati della query, un record per riga

13.4 Eseguire le query

Indice

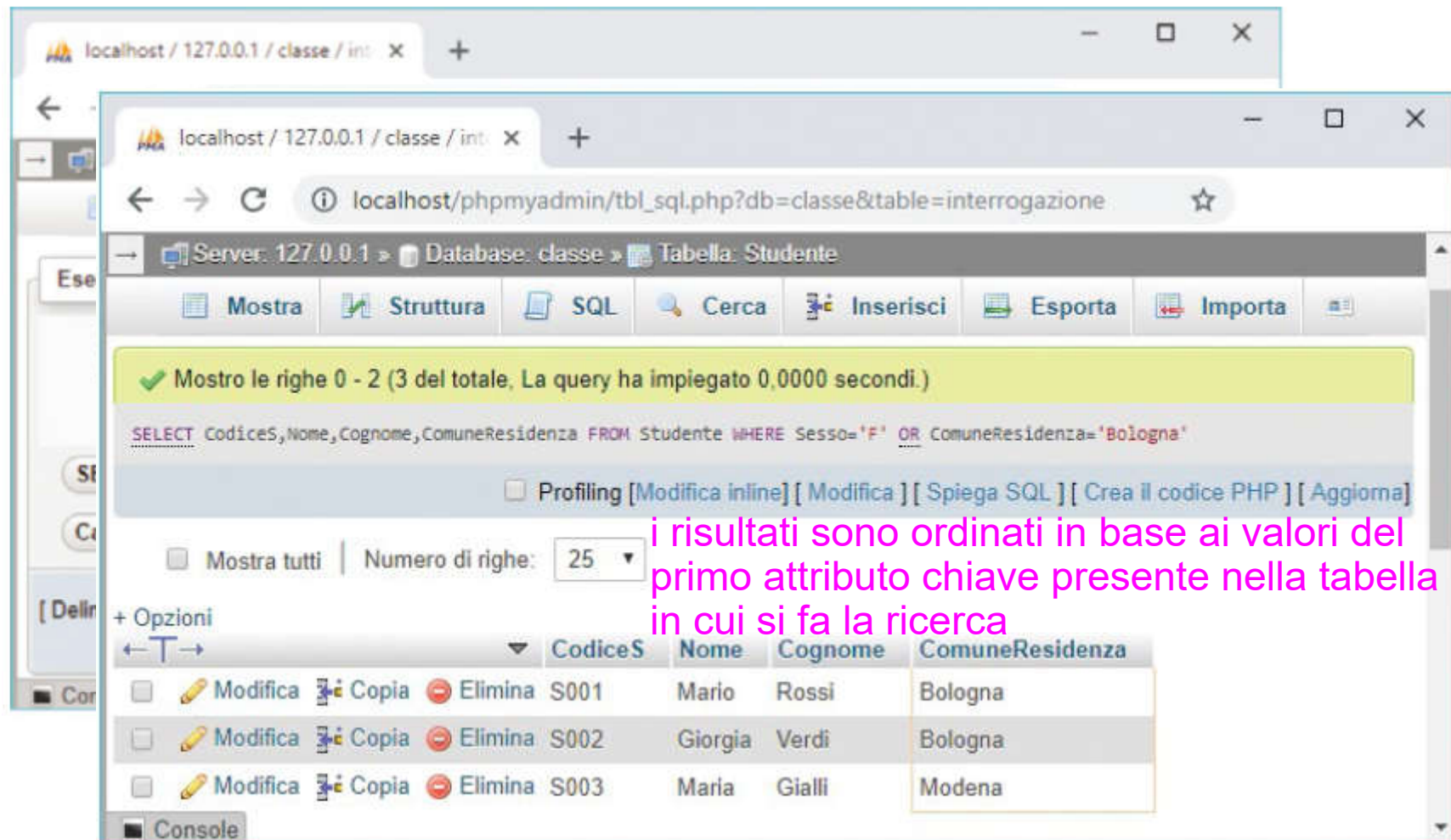
In phpMyAdmin le **query** si eseguono nella **scheda SQL**.



13.4 Eseguire le query

Indice

In phpMyAdmin le **query** si eseguono nella **scheda SQL**.



The screenshot shows the phpMyAdmin interface with the SQL tab selected. The query executed is: `SELECT CodiceS, Nome, Cognome, ComuneResidenza FROM Studente WHERE Sesso='F' OR ComuneResidenza='Bologna'`. The result shows 3 rows, ordered by the first primary key attribute (CodiceS). A pink annotation points to the 'CodiceS' column header, stating: "i risultati sono ordinati in base ai valori del primo attributo chiave presente nella tabella in cui si fa la ricerca".

Mostrare le righe 0 - 2 (3 del totale, La query ha impiegato 0,0000 secondi.)

```
SELECT CodiceS, Nome, Cognome, ComuneResidenza FROM Studente WHERE Sesso='F' OR ComuneResidenza='Bologna'
```

☐ Profiling [Modifica inline] [Modifica] [Spiega SQL] [Crea il codice PHP] [Aggiorna]

☐ Mostra tutti | Numero di righe: 25

	CodiceS	Nome	Cognome	ComuneResidenza
☐ Modifica ☐ Copia ☐ Elimina	S001	Mario	Rossi	Bologna
☐ Modifica ☐ Copia ☐ Elimina	S002	Giorgia	Verdi	Bologna
☐ Modifica ☐ Copia ☐ Elimina	S003	Maria	Gialli	Modena

Console

13.4 Eseguire le query

Indice

In phpMyAdmin le **query** si eseguono nella **scheda SQL**.

CodiceS	Nome	Cognome	ComuneResidenza
S001	Mario	Rossi	
S002	Giorgia	Verdi	
S003	Maria	Gialli	

- Clicca per ordinare i risultati secondo questa colonna.

- Shift+Click per aggiungere questa colonna alla clausola ORDER BY o per alternare ASC/DESC.

- Ctrl+Click o Alt+Click (Mac: Shift+Option+Click) per rimuovere la colonna dalla clausola ORDER BY

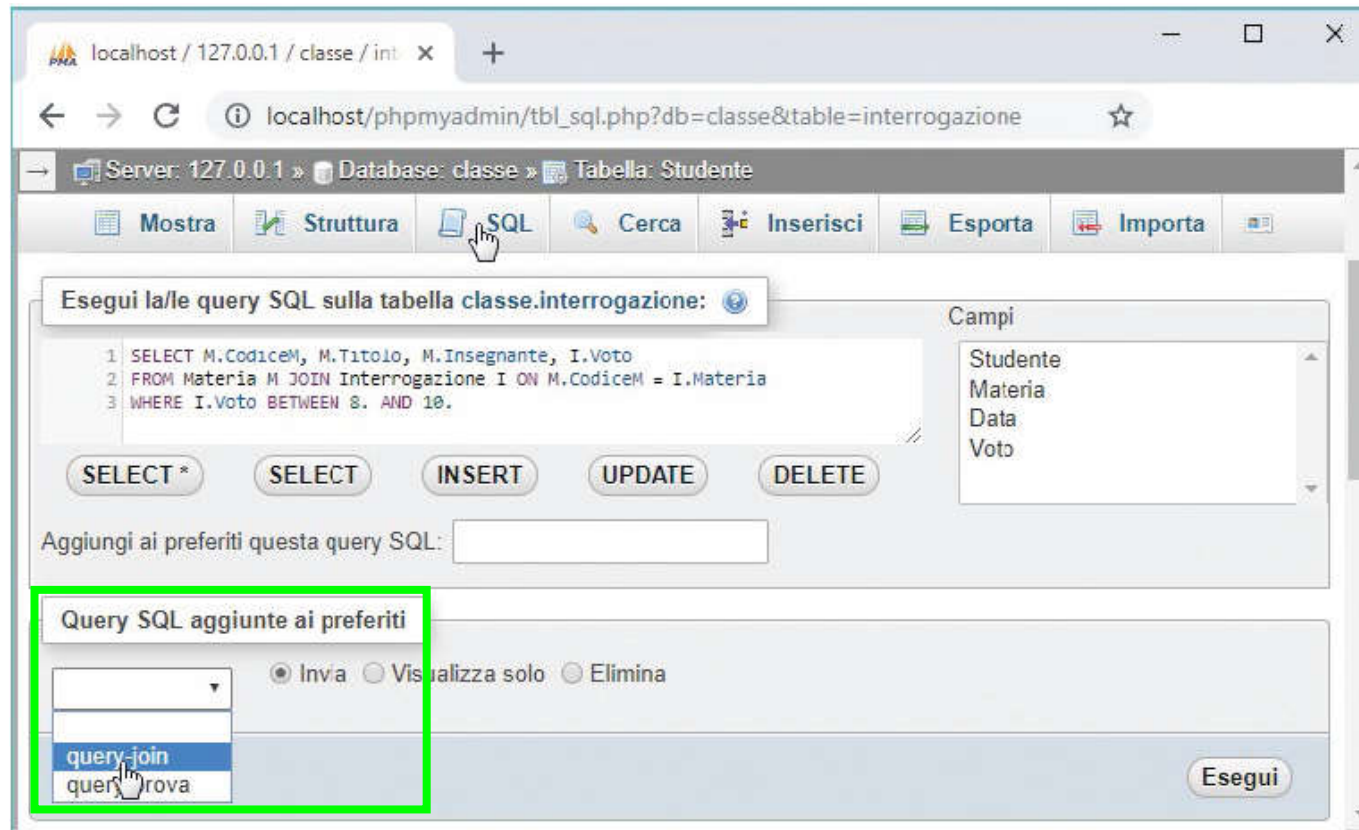
CodiceS	Nome	Cognome	▲ 1	ComuneResidenza
S003	Maria	Gialli		Modena
S001	Mario	Rossi		Bologna
S002	Giorgia	Verdi		Bologna

Un vantaggio dell' interfaccia grafica è l'estrema facilità con cui si possono **riordinare i risultati delle query** in base a qualsiasi campo del database: basti un clic sull' intestazione della colonna.

13.4 Eseguire le query

Indice

In phpMyAdmin le **query** si eseguono nella **scheda SQL**.



È anche possibile **memorizzare le query**, così da poterle riutilizzare in seguito senza doverle riscrivere.